arXiv:2007.04833v2 [cs.IR] 9 Jun 2021

Towards Open-World Recommendation: An Inductive Model-based Collaborative Filtering Approach

Qitian Wu [1 2] **Hengrui Zhang** [1] **Xiaofeng Gao** [1 2] **Junchi Yan** [1 2] **Hongyuan Zha** [3]

Abstract

Recommendation models can effectively estimate underlying user interests and predict one's future behaviors by factorizing an observed user-item rating matrix into products of two sets of latent factors. However, the user-specific embedding factors can only be learned in a transductive way, making it difficult to handle new users on-the-fly. In this paper, we propose an inductive collaborative filtering framework that contains two representation models. The first model follows conventional matrix factorization which factorizes a group of key users' rating matrix to obtain *meta latents*. The second model resorts to attention-based structure learning that estimates hidden relations from query to key users and learns to leverage meta latents to inductively compute embeddings for query users via neural message passing. Our model enables inductive representation learning for users and meanwhile guarantees equivalent representation capacity as matrix factorization. Experiments demonstrate that our model achieves promising results for recommendation on few-shot users with limited training ratings and new unseen users which are commonly encountered in open-world recommender systems.

1. Introduction

As information explosion has become one major factor affecting human life, recommender systems, which can filter useful information and contents of user's potential interests, play an increasingly indispensable role. Recommendation problems can be generally formalized as matrix completion (MC) where one has a user-item rating matrix whose entries, which stand for interactions of users with items (ratings or click behaviors), are partially observed. The goal of MC is to predict missing entries (unobserved or future potential interactions) in the matrix based on the observed ones.

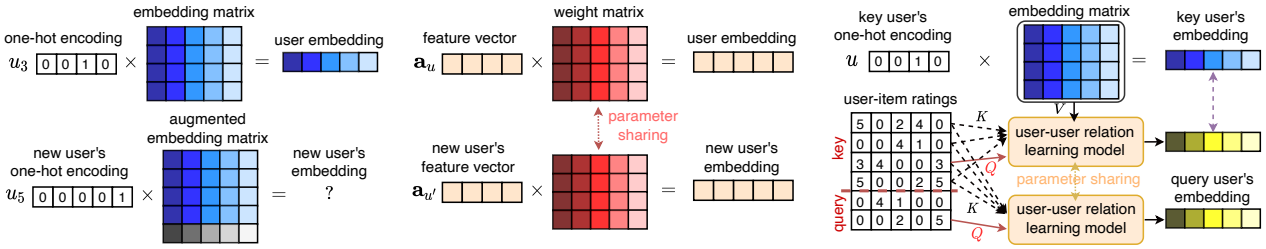
Modern recommender systems need to meet two important requirements for practical utility. First of all, *recommendation models should have enough expressiveness to capture diverse user interests and preferences so that the systems can accomplish personalized recommendation*. Existing methods based on collaborative filtering (CF)[1] or, interchangeably, matrix factorization (MF) have shown great power in this problem by factorizing the rating matrix into two classes of latent factors (i.e., embeddings) for users and items respectively, and further leverage dot-product of two factors to predict potential ratings (Hu et al., 2008; Koren et al., 2009; Rendle et al., 2009; Srebro et al., 2004; Zheng et al., 2016). Equivalently, for each user, the methods consider a one-hot user encoding as input, and assume a user-specific embedding function mapping user index to a latent factor. Such learnable latent factors can represent user's preferences in a low-dimensional space. Recent works extend MF with complex architectures, like multi-layer perceptrons (Dziugaite & Roy, 2015), recurrent units (Monti et al., 2017), autoregressive models (Zheng et al., 2016), graph neural networks (van den Berg et al., 2017), etc., achieving state-of-the-art results on both benchmark datasets and commercial systems (Covington et al., 2016; Ying et al., 2018; Kang et al., 2020).

The second requirement stems from a key observation from real-world scenarios: *recommender systems often interact with a dynamic open world where new users, who are not exposed to models during training, may appear in test stage*. This requires that models trained on a set of users manage to adapt to unseen users. However, the above-mentioned CF models would fail in this situation since the embedding factors are parameterized for specific users and need to be learned collaboratively with all other users in transductive setting (see Fig. 1 for illustration). Brute-force ways include:

[1]Department of Computer Science and Engineering, Shanghai Jiao Tong University. [2]MoE Key Lab of Artificial Intelligence, AI Institute, Shanghai Jiao Tong University. [3]School of Data Science, Shenzhen Institute of Artificial Intelligence and Robotics for Society, The Chinese University of Hong Kong, Shenzhen. Correspondence to: Xiaofeng Gao <gao-xf@cs.sjtu.edu.cn>.

Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).

[1]In recent literature, collaborative filtering (CF) approaches often refer to model-based CF, i.e. matrix factorization, while its memory-based counterpart, as an heuristic approach, adopts similarity methods like KNN for recommendation.



(a) Collaborative filtering model (b) Feature-drive model (c) Inductive CF model (ours)

Figure 1. (a) Conventional (model-based) collaborative filtering model assumes user-specific embeddings for user representation, which is parameterized in a user-specific manner and limits the model's capability to handle new unseen users. (b) Feature-driven models manage to deal with new users and achieves inductive learning via modeling a user-sharing mapping from user features to representations, but would lack enough expressiveness for diverse user interests. (c) Our proposed inductive CF model that absorbs the advantage of both of the worlds, achieving inductive representation learning for users without compromising representation capacity.

1) retrain a model with an augmented rating matrix; 2) consider incremental learning (Hu et al., 2008) for new users' embeddings. The former requires much extra time cost, which would be unacceptable for online systems, while the latter is prone for over-fitting and disables on-the-fly inference. There are quite a few studies that propose inductive matrix completion models using user features (Jain & Dhillon, 2013; Xu et al., 2013; Cheng et al., 2016; Ying et al., 2018; Zhong et al., 2018). Their different thinking paradigm, as shown in Fig. 1(b), is to target a user-sharing mapping from user features to user representations, instead of from one-hot user indices. Since the feature space is shared among users, such methods are able to adapt a trained model to unseen users. Nevertheless, feature-driven models may suffer from limited expressiveness with low-quality features that have weak correlation with labels. For example, users with the same age and occupation (commonly used features) may have distinct ratings on movies. Unfortunately, high-quality features that can unveil user interests for personalized recommendation are often hard to collect due to increasingly concerned privacy issues.

A following question arises: *Can we build a recommendation model that guarantees enough expressiveness for personalized preferences and enables inductive learning?* Such question still remains unexplored so far. In fact, simultaneously meeting the two requirements is a non-trivial challenge when high-quality user features are unavailable. First, to achieve either of them, one often needs to compromise on the other. The user-specific embedding vectors, which assume independent parametrization for different users, can give sufficient capacity for learning distinct user preferences from historical rating patterns (Mikolov et al., 2013). To make inductive learning possible, one needs to construct a shared input feature space among users out of the rating matrix, as an alternative to one-hot user encodings. However, the new constructed features have relatively insufficient expressive power. Second, the computation based on new feature space often brings extra costs for time and space, which limits model's scalability on large-scale datasets.

In this paper, we propose an InDuctive Collaborative Filtering model (IDCF)[2] as a general CF framework that achieves inductive learning for user representations and meanwhile guarantees enough expressiveness and scalability, as shown in Fig. 1. Our approach involves two representation models. A conventional matrix factorization model that factorizes a group of key users' rating matrix to obtain their user-specific embeddings, which we call *meta latents*. On top of that, we further design a relation learning model, specified as multi-head attention mechanism, that learns hidden graphs between key and query users w.r.t. their historical rating behaviors. The uncovered relation graphs enable neural message passing among users in latent space and inductive computation of user-specific representations for query users.

Furthermore, we develop two training strategies for practical cases in what we frame as open-world recommendation: inductive learning for interpolation and inductive learning for extrapolation, respectively. In the first case, query users are disjoint from key users in training and the model is expected to provide decent performance on few-shot query users with limited training ratings. In the second case, query users are the same as key users in training and the trained model aims to tackle zero-shot test users that are unseen before. We show that our inductive model guarantees equivalent capacity as matrix factorization and provides superior expressiveness compared to other inductive models (feature-driven, item-based and graph-based). Empirically, we conduct experiments on five datasets for recommendation (with both explicit and implicit feedbacks). The comprehensive evaluation demonstrates that IDCF 1) consistently outperform various inductive models by a large margin on recommendation for few-shot users , and 2) can achieve superior results on recommendation for new unseen users (with few historical ratings not used in training). Moreover, compared with transductive models, IDCF provides very close reconstruction error and can even outperform them when training

[2]The codes are available at https://github.com/qitianwu/IDCF.

ratings becomes scarce. The contributions of this paper are summarized as follows.

1) We propose a new inductive collaborative filtering framework that can inductively compute user representations for new users based on a set of pretrained meta latents, which is suitable for open-world recommender systems. The new approach can serve as a brand new learning paradigm for inductive representation learning.

2) We show that a general version of our model can minimize reconstruction loss to the same level as vanilla matrix factorization model under a mild condition. Empirically, IDCF gives very close RMSE to transductive CF models.

3) IDCF achieves very competitive and superior RMSE/NDCG/AUC results on few-shot and new unseen users compared with various inductive models on explicit feedback and implicit feedback data.

As a general model-agnostic framework, IDCF can flexibly incorporate with various off-the-shelf MF models (e.g. MLP-based, GNN-based, RNN-based, attention-based, etc.) as backbones as well as combine with user profile features for hybrid recommendation.

2. Background

Consider a general matrix completion (MC) problem which deals with a user-item rating matrix $R = \{r_{ui}\}_{M \times N}$ where M and N is the number of users and items, respectively. For explicit feedback, r_{ui} records rating value of user u on item i. For implicit feedback, r_{ui} is a binary entry for whether user u rated (or clicked on, reviewed, liked, purchased, etc.) item i or not. The recommendation problem can be generally formalized as: given partially observed entries in R, one needs to estimate the missing values in the matrix.

Existing recommendation models are mostly based on collaborative filtering (CF) or, interchangeably, matrix factorization (MF) where user u (resp. item i) corresponds to a d-dimensional latent factor (i.e., embedding) $\mathbf{p}_u$ (resp. $\mathbf{q}_i$). Then one has a prediction model $\hat{r}_{ui} = f(\mathbf{p}_u, \mathbf{q}_i)$ where f can be basically specified as simple dot product or some complex architectures, like neural networks, graph neural networks, etc. One advantage of CF models is that the user-specific embedding $\mathbf{p}_u$ (as learnable parameters) can provide enough expressive power for learning diverse personal preferences from user historical behaviors and decent generalization ability through collaborative learning with all the users and items. However, such user-specific parametrization limits the model in transductive learning. In practical situations, one cannot have information for all the users that may appear in the future when collecting training data. When it comes to new users in test stage, the model has to be retrained and cannot deliver on-the-fly inference.

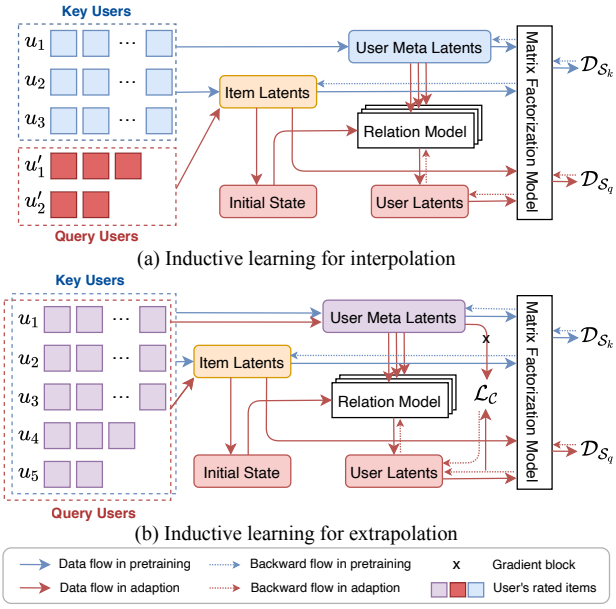


Figure 2. Framework of inductive collaborative filtering for open-world recommendation. We consider two scenarios, inductive learning for interpolation (query users are different from key users) and extrapolation (query users are the same as key users), which aims to handle few-shot query users and zero-shot new users in test stage. In both cases, the learning procedures contain pretraining and adaption. The pretraining learns initial user meta latents via matrix factorization over key users' ratings. The adaption optimizes a relation model, which estimates hidden relations from key to query users and inductively computes embeddings for query users. In particular, in extrapolation case, we introduce a self-supervised contrastive loss that enforces similarity between meta latents and inductively computed embeddings for the same users.

3. Methodology

We propose the InDuctive Collaborative Filtering (IDCF) model. Our high-level methodology stems from a key observation: there exist a (or multiple) latent relational graph among users that represents preference proximity and behavioral interactions. For instance, social networks and following networks in social media can be seen as realizations of such relational graphs, but in most cases, the graph structures are unobserved and implicitly affect user's behaviors. If we can identify the graph structures, we can leverage the idea of message passing (Scarselli et al., 2009; Hamilton et al., 2017; Gilmer et al., 2017; Chen et al., 2020), propagating learned embeddings from one group of users to others, especially, in an inductive manner.

We formulate our model through two sets of users: 1) *key users* (denoted by $\mathcal{U}_k$), for which we learn their embeddings by matrix factorization and use them as *meta latents*; 2) *query users* (denoted by $\mathcal{U}_q$), for which we consider neural message passing to inductively compute their embeddings. Assume $|\mathcal{U}_k| = M_k$ and $|\mathcal{U}_q| = M_q$. Correspondingly, we have two rating matrices: $R_k = \{r_{ui}\}_{M_k \times N}$ (given by $\mathcal{U}_k$)

and $R_q = \{r_{u'i}\}_{M_q \times N}$ (given by $\mathcal{U}_q$). Based on this, we further consider two scenarios.

Scenario I: Inductive learning for interpolation. $\mathcal{U}_1 \cap \mathcal{U}_2 = \emptyset$, i.e., query users are disjoint from key users. In training, it learns to leverage meta latents of key users to compute representations for another group of query users in a supervised way. The model is expected to perform robustly on few-shot query users with limited training ratings.

Scenario II: Inductive learning for extrapolation. $\mathcal{U}_1 = \mathcal{U}_2$, i.e., key and query users are the same. In training, it learns to use meta latents given by key users to represent themselves in a self-supervised way. Then the trained model aims to deal with zero-shot test users (with limited observed ratings not used in training) that are unseen before.

Notice that in the above two cases, we assume no side information, such as user profile features (ages, occupation, etc,), social networks, item content features, etc., besides the observed user-item rating matrix. We frame the problem as open-world recommendation which requires the model to deal with few-shot and zero-shot users. We present our model framework for two settings in Fig. 2 and go into the details in the following.

3.1. Matrix Factorization Model

We first pretrain a (transductive) matrix factorization model for $\mathcal{U}_k$ using R_k, denoted as $\hat{r}_{ui} = f_\theta(\mathbf{p}_u, \mathbf{q}_i)$, where $\mathbf{p}_u \in \mathbb{R}^d$ denotes a user-specific embedding for user u in $\mathcal{U}_k$, $\mathbf{q}_i \in \mathbb{R}^d$ denotes an item-specific embedding for item i and f_θ can be simple dot-product or a network with parameter θ. Section 5.1 gives details for two specifications for f_θ using neural network and graph convolution network, as used in our implementation. Denote $\mathbf{P}_k = \{\mathbf{p}_u\}_{M_k \times d}$, $\mathbf{Q} = \{\mathbf{q}_i\}_{N \times d}$ and the objective becomes

$$\min_{\mathbf{P}_k, \mathbf{Q}, \theta} \mathcal{D}_{\mathcal{S}_k}(\hat{R}_k, R_k), \tag{1}$$

where we define $\hat{R}_k = \{\hat{r}_{ui}\}_{M_k \times N}$, $\mathcal{D}_{\mathcal{S}_k}(\hat{R}_k, R_k) = \frac{1}{T_k} \sum_{(u,i) \in \mathcal{S}_k} l(r_{ui}, \hat{r}_{ui})$ and $\mathcal{S}_k \in ([M_k] \times [N])^{T_k}$ is a set with size T_k containing indices of observed entries in R_k. Here $l(r_{ui}, \hat{r}_{ui})$ can be MSE loss for explicit user feedbacks or cross-entropy loss for implicit user feedbacks.

We treat the pretrained factors $\mathbf{P}_k$ as *meta latents*, to inductively compute user latents for query via a relation model.

3.2. Inductive Relation Model

Assume $\mathbf{C} = \{c_{uu'}\}_{M_k \times M_q}$, where $c_{uu'} \in \mathbb{R}$ denotes weighted edge from user $u \in \mathcal{U}_k$ to user $u' \in \mathcal{U}_q$, and define $\mathbf{c}_{u'} = [c_{1u'}, c_{2u'}, \cdots c_{M_k u'}]^\top$ the u'-th column of $\mathbf{C}$. Then we express embedding of user u' as $\tilde{\mathbf{p}}_{u'} = \mathbf{c}_{u'}^\top \mathbf{P}_k$, a weighted sum of embeddings of key users. The rating can be predicted by $\hat{r}_{u'i} = f_\theta(\tilde{\mathbf{p}}_{u'}, \mathbf{q}_i)$ and the problem of model optimization becomes:

$$\min_{\mathbf{C}, \mathbf{Q}} \mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q), \tag{2}$$

where we define $\hat{R}_q = \{\hat{r}_{u'i}\}_{M_q \times N}$, $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q) = \frac{1}{T_q} \sum_{(u',i) \in \mathcal{S}_q} l(r_{u'i}, \hat{r}_{u'i})$ and $\mathcal{S}_q \in ([M_q] \times [N])^{T_q}$ is a set with size T_q containing indices of observed entries in R_q. The essence of above method is taking attentive pooling as message passing from key to query users. We first justify this idea by analyzing its capacity and then propose a parameterized model enabling inductive learning.

Theoretical Justification If we use dot-product for f_θ in the MF model, then we have $\hat{r}_{u'i} = \tilde{\mathbf{p}}_{u'}^\top \mathbf{q}_i$. We compare Eq. (2) with using matrix factorization over R_q:

$$\min_{\tilde{\mathbf{P}}_q, \mathbf{Q}} \mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q), \tag{3}$$

where $\tilde{\mathbf{P}}_q = \{\tilde{\mathbf{p}}_{u'}\}_{M_q \times d}$ and have the following theorem.

Theorem 1. *Assume Eq. (3) can achieve $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q) < \epsilon$ and the optimal $\mathbf{P}_k$ given by Eq. (1) satisfies column-full-rank, then there exists at least one solution for $\mathbf{C}$ in Eq. (2) such that $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q) < \epsilon$.*

The only condition that $\mathbf{P}_k$ is column-full-rank can be trivially guaranteed since $d \ll N$. The theorem shows that the proposed model can minimize the reconstruction loss of MC to at least the same level as matrix factorization which gives sufficient capacity for learning personalized user preferences from historical rating patterns.

Parametrization We have shown that using attentive pooling does not sacrifice model capacity than MF/CF models under a mild condition. However, directly optimizing over $\mathbf{C}$ is intractable due to its $O(M_k M_q)$ parameter space and $\mathbf{c}_{u'}$ is a user-specific high-dimensional vector which disallows inductive learning. Hence, we parametrize $\mathbf{C}$ with an attention network, reducing parameters and enabling it for inductive learning. Concretely, we estimate the adjacency score between user u' and u as

$$c_{u'u} = \frac{\mathbf{e}^\top [\mathbf{W}_q \mathbf{d}_{u'} \oplus \mathbf{W}_k \mathbf{p}_u]}{\sum_{u_0 \in \mathcal{U}_1} \mathbf{e}^\top [\mathbf{W}_q \mathbf{d}_{u'} \oplus \mathbf{W}_k \mathbf{p}_{u_0}]}, \tag{4}$$

where $\mathbf{e} \in \mathbb{R}^{2d \times 1}$, $\mathbf{W}_q \in \mathbb{R}^{d \times d}$, $\mathbf{W}_k \in \mathbb{R}^{d \times d}$ are trainable parameters, $\oplus$ denotes concatenation and $\mathbf{d}_{u'} = \sum_{i \in \mathcal{I}_{u'}} \mathbf{q}_i$. Here $\mathcal{I}_{u'} = \{i | r_{u'i} > 0\}$ includes the historically rated items of user u'. The attention network captures first-order user proximity on behavioral level and also maintains second-order proximity that users with similar historical ratings on items would have similar relations to other users. Besides, if $\mathcal{I}_{u'}$ is empty (for extreme cold-start recommendation), we can randomly select a group of items from all the candidates. Also, if user's profile features are available, we

can harness them as $\mathbf{d}_{u'}$. We provide details in Appendix C. Yet, in the main body of our paper, we focus on learning from user-item rating matrix, i.e., the common setting for CF approaches.

The normalization in Eq. (4) requires computation for all the key users, which limits scalability to large dataset. Therefore, we use sampling strategy to control the size of key users in relation graph for each query user and further consider multi-head attentions that independently sample different subsets of key users. The attention score given by the l-th head is

$$c_{u'u}^{(l)} = \frac{(\mathbf{e}^{(l)})^\top [\mathbf{W}_q^{(l)} \mathbf{d}_{u'} \oplus \mathbf{W}_k^{(l)} \mathbf{p}_u]}{\sum_{u_0 \in \mathcal{U}_k^{(l)}} (\mathbf{e}^{(l)})^\top [\mathbf{W}_q^{(l)} \mathbf{d}_{u'} \oplus \mathbf{W}_k^{(l)} \mathbf{p}_{u_0}]}, \quad (5)$$

where $\mathcal{U}_k^{(l)}$ denotes a subset of key users sampled from $\mathcal{U}_k$. Each attention head independently aggregates embeddings of different subsets of key users and the final inductive representation for user u' can be given as

$$\tilde{\mathbf{p}}_{u'} = \mathbf{W}_o \left[\bigoplus_{l=1}^{L} \sum_{u \in \mathcal{U}_k^{(l)}} c_{u'u}^{(l)} \mathbf{W}_v^{(l)} \mathbf{p}_u \right], \quad (6)$$

where $\mathbf{W}_o \in \mathbb{R}^{d \times Ld}$ and $\mathbf{W}_v^{(l)} \in \mathbb{R}^{d \times d}$. To keep the notation clean, we denote $\tilde{\mathbf{p}}_{u'} = h_w(\mathbf{d}_{u'})$ and $w = \cup_{l=1}^{L} \{\mathbf{e}^{(l)}, \mathbf{W}_q^{(l)}, \mathbf{W}_k^{(l)}, \mathbf{W}_v^{(l)}\} \cup \{\mathbf{W}_o\}$.

With fixed meta latents $\mathbf{P}_1$ and $\mathbf{Q}$, we can consider optimization for our inductive relation model

$$\min_{w,\theta} \mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q). \quad (7)$$

We found that using fixed $\mathbf{Q}$ here contributes to much better performance than optimizing it in the second stage.

Next we analyze the generalization ability of inductive model on query users. Also, consider f_θ as dot-product operation and we assume $c_{u'u} \in \mathbb{R}^+$ to simplify the analysis. Now, we show that the generalization error $\mathcal{D}(\hat{R}_q, R_q) = \mathbb{E}_{(u',i)}[l(r_{u'i}, \hat{r}_{u'i})]$ on query users is bounded by the numbers of key users and observed ratings of query users.

Theorem 2. *Assume 1) $\mathcal{D}$ is L-Lipschitz, 2) for $\forall \hat{r}_{u'i} \in \hat{R}_q$ we have $|\hat{r}_{u'i}| \leq B$, and 3) the L1-norm of $\mathbf{c}_{u'}$ is bounded by H. Then with probability at least $1-\delta$ over the random choice of $\mathcal{S}_q \in ([M_q] \times [N])^{T_q}$, it holds that for any $\hat{R}_q$, the gap between $\mathcal{D}(\hat{R}_q, R_q)$ and $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q)$ will be bounded by*

$$O\left(2LHB\sqrt{\frac{2M_q \ln M_k}{T_q}} + \sqrt{\frac{ln(1/\delta)}{T_q}}\right). \quad (8)$$

The theorem shows that the generalization error bound depends on the size of $\mathcal{U}_k$. Theorem 1 and 2 show that the configuration of $\mathcal{U}_k$ has an important effect on model capacity and generalization ability. On one hand, we need to make key users in $\mathcal{U}_k$ 'representative' of diverse user behavior patterns on item consumption in order to guarantee enough representation capacity. Also, we need to control the size of $\mathcal{U}_k$ to maintain generalization ability.

3.3. Model Optimization

The complete training process is comprised of: *pretraining* and *adaption*. In the first stage, we train a MF model in transductive setting via Eq. (1) and obtain embeddings $\mathbf{P}_k$, $\mathbf{Q}$ and network f_θ. The adaption involves optimization for the inductive relation model h_w and finetuning the prediction network f_θ via Eq. (3). In particular, in terms of inductive learning for extrapolation (i.e., $\mathcal{U}_k = \mathcal{U}_q$), we further consider a self-supervised contrastive loss that pursuits similarity between the inductively computed user embeddings and the ones given by the MF model. Concretely, the loss is defined as

$$\min_{w,\theta} \mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q) + \lambda \mathcal{L}_C(\mathbf{P}_k, \tilde{\mathbf{P}}_q), \quad (9)$$

where λ is a trading-off hyper-parameter and

$$\mathcal{L}_C(\mathbf{P}_k, \tilde{\mathbf{P}}_q) = \frac{1}{M_q} \sum_u \log \left(\frac{\exp(\mathbf{p}_u^\top \tilde{\mathbf{p}}_u)}{\sum_{u' \in \mathcal{U}_q} \exp(\mathbf{p}_u^\top \tilde{\mathbf{p}}_{u'})} \right). \quad (10)$$

The summation in the denominator can be approximated by in-batch negative samples with mini-batch training.

4. Comparison with Existing Works

We discuss related works and highlight our differences.

Feature-driven Recommendation. The collaborative filtering (CF) models do not assume any side information other than the rating matrix, but they cannot be trained in inductive ways due to the learnable user-specific embedding $\mathbf{p}_u$. To address the issue, previous works leverage side information, e.g. user profile features, to achieve inductive learning (Jain & Dhillon, 2013; Xu et al., 2013; Cheng et al., 2016; Ying et al., 2018; Zhong et al., 2018). Define user features (like age, occupation) as $\mathbf{a}_u$ and item features (like movie genre, director) as $\mathbf{b}_i$. The feature-driven model targets a prediction model $\hat{r}_{ui} = g(\mathbf{a}_u, \mathbf{b}_i)$. Since the space of $\mathbf{a}_u$ is shared among users, a model trained on one group of users can adapt to other users without retraining. However, feature-driven models often provide limited performance since the shared feature space is not expressive enough compared to user-specific embedding factors (see Appendix A for more discussions). Another issue is that high-quality features are hard to collect in practice. A key advantage of IDCF is the capability for inductive representation learning without using features.

Table 1. Statistics of five datasets used in our experiments. Amazon-Books and Amazon-Beauty datasets contain implicit user feedbacks while Douban, ML-100K, ML-1M have explicit feedbacks (ratings range within $[1, 2, 3, 4, 5]$).

Dataset	# Users	#Items	# Ratings	Density	# Key/Query Users	# Training/Test Instances
Douban	3,000	3,000	0.13M	0.0152	2,131/869	80,000/20,000
Movielens-100K	943	1,682	0.10M	0.0630	123,202/13,689	
Movielens-1M	6,040	3,706	1.0M	0.0447	5,114/926	900,199/100,021
Amazon-Books	52,643	91,599	2.1M	0.0012	49,058/3,585	2,405,036/526,430
Amazon-Beauty	2,944	57,289	0.08M	0.0004	780/2,164	53,464/29,440

Inductive Matrix Completion. There are few existing works that attempt to handle inductive matrix completion using only user-item rating matrix. (Hartford et al., 2018) (F-EAE) puts forward an exchangeable matrix layer that takes a whole rating matrix as input and inductively outputs prediction for missing ratings. However, the scalability of F-EAE is limited since it requires the whole rating matrix as input for training and inference for users, while IDCF enables mini-batch training and efficient inference. Besides, (Zhang & Chen, 2020) (IGMC) proposes to use local subgraphs of user-item pairs in a bipartite graph of rating information as input features and further adopt graph neural networks to encode subgraph structures for rating prediction. The model achieves inductive learning via replacing users' one-hot indices by shared input features (i.e., index-free local subgraph structures). However, the expressiveness of IGMC is limited since the local subgraph structures can be indistinguishable for users with distinct behaviors (see Appendix A for more discussions), and the issue would become worse for implicit feedback data. By contrast, IDCF has equivalent expressiveness as original CF models. Another drawback of F-EAE and IGMC is that their models cannot output user representations. Differently, IDCF maintains the ability to give user-specific representations, which reflect users' preferences and can be used for downstream tasks (like user behavior modeling (Liu et al., 2020), user-controllable recommendation (Ma et al., 2019; Cen et al., 2020), target advertisement and influence maximization (Khalil et al., 2017; Manchanda et al., 2019), etc.).

Item-based CF Models. Previous works use item embeddings as representation for users. (Cremonesi et al., 2010; Kabbur et al., 2013) adopts a combination of items rated by users to compute user embeddings and frees the model from learning parametric user-specific embeddings. Furthermore, there are quite a few auto-encoder architectures for recommendation problem, leveraging user's rating vector (ratings on all the items) as input, estimating user embedding (as latent variables), and decoding missing values in the rating vector (Sedhain et al., 2015; Liang et al., 2018). With item embeddings and user's rating history, these methods enable inductive learning for user representation and can adapt to new users on-the-fly. On methodological level, IDCF has the following differences. First, IDCF assumes learnable embeddings for both users and items, which maintains equivalent representation capacity as general MF/CF models (as proved in Theorem 1). Item-based models only consider learnable embeddings for items, and may suffer from limited representation capacity. Second, IDCF learns message-passing graphs among users by a relation model to obtain better user representations, instead of directly aggregating a given observed set of items' embeddings as item-based models.

5. Experiment

In this section, we apply the proposed model IDCF to several real-world recommendation datasets to verify and dissect its effectiveness. Before going into the experiment results, we first introduce experiment setup including dataset information, evaluation protocol and implementation details.

5.1. Experiment Setups

Datasets. We consider five common recommendation benchmarks: *Douban*, *Movielens-100K (ML-100K)*, *Movielens-1M (ML-1M)*, *Amazon-Books* and *Amazon-Beauty*. Douban, ML-100K and ML-1M have explicit user's ratings on movies. Amazon-Books and Amazon-Beauty contain implicit user feedbacks (the records of user's interactions with items). For Douban and ML-100K, we use the training/testing splits provided by (Monti et al., 2017). For ML-1M[3], we follow previous works (van den Berg et al., 2017; Hartford et al., 2018; Zhang & Chen, 2020) and use 9:1 training/testing spliting. For two Amazon datasets, we use the last ten interactions of each user for test and the remaining for training. We leave out 5% training data as validation set for early stopping in training. Note that the raw datasets for Amazon-Books and Amazon-Beauty [4] are very large and sparse ones and we filter out infrequent items and users with less than five ratings. The statistics of datasets used in our experiments are summarized in Table 1.

Evaluation. For datasets with explicit feedbacks, the goal is to predict user's ratings on items, i.e. estimate the missing values in user-item rating matrix. The task can be seen as a multi-class classification or regression problem. We use

[3]https://grouplens.org/datasets/movielens/

[4]http://jmcauley.ucsd.edu/data/amazon/

Table 2. Test RMSE and NDCG for all the users (All) and few-shot users (FS) in Douban, ML-100K and ML-1M. We highlight the best scores among all the (resp. inductive) models with bold (resp. underline). *Inductive* indicates whether the method can achieve inductive learning. *Feature* indicates whether the method relies on user features.

Method	Inductive	Feature	Douban				ML-100K				ML-1M			
			RMSE		NDCG		RMSE		NDCG		RMSE		NDCG	
			All	FS	All	FS	All	FS	All	FS	All	FS	All	FS
PMF	No	No	0.737	0.718	0.939	0.954	0.932	1.003	0.858	0.843	0.851	0.946	0.919	**0.940**
NNMF	No	No	0.729	**0.705**	0.939	0.952	0.925	0.987	0.895	0.878	0.848	**0.940**	0.920	0.937
GCMC	No	No	0.731	0.706	0.938	**0.956**	0.911	0.989	0.900	**0.886**	**0.837**	0.947	0.923	0.939
NIMC	Yes	Yes	0.732	0.745	0.928	0.931	1.015	1.065	0.832	0.824	0.873	0.995	0.889	0.904
BOMIC	Yes	Yes	0.735	0.747	0.923	0.925	0.931	1.001	0.828	0.815	0.847	0.953	0.905	0.924
F-EAE	Yes	No	0.738	-	-	-	0.920	-	-	-	0.860	-	-	-
IGMC	Yes	No	**0.721**	0.728	-	-	**0.905**	0.997	-	-	0.857	0.956	-	-
IDCF-NN (ours)	Yes	No	0.738	0.712	0.939	**0.956**	0.931	0.996	0.896	0.880	0.844	0.952	0.922	**0.940**
IDCF-GC (ours)	Yes	No	0.733	0.712	**0.940**	**0.956**	**0.905**	**0.981**	**0.901**	0.884	0.839	0.944	**0.924**	**0.940**

RMSE and NDCG to evaluate general reconstruction error and personalized ranking performance. RMSE counts the overall l2 distance from predicted ratings to the ground-truth, while NDCG is an averaged score that measures the consistency between the ranking of predicted ratings and that of the ground-truth for each user. For datasets with implicit feedbacks, the goal is to predict whether a user interacts with an item. The task is essentially a one-class classification problem. Since the dataset is very sparse and only has positive instances, we uniformly sample five items as negative samples for each clicked item and adopt AUC and NDCG to measure the global and personalized ranking accuracy, respectively. AUC is short of Area Under of the ROC Curve which measures the global consistency between the ranking of all the predicted user-item interactions and the ground-truth (which ranks all the 1's before 0'). More details for evaluation metrics are provided in Appendix D.2

Implementations. We consider two specifications for f_θ in our model: IDCF-NN, which adopts multi-layer perceptron for f, and IDCF-GC, which uses graph convolution network for f.

Feedforward Neural Network as Matrix Factorization Model (IDCF-NN). We follow the architecture in NNMF (Dziugaite & Roy, 2015) and use neural network for f_θ. Here we combine a three-layer neural network and a shallow dot-product operation. Concretely,

$$f_\theta(\mathbf{p}_u, \mathbf{q}_i) = \frac{(\mathbf{p}_u^\top \mathbf{q}_i + nn([\mathbf{p}_u \| \mathbf{q}_i \| \mathbf{p}_u \odot \mathbf{q}_i]))}{2} + b_u + b_i, \tag{11}$$

where nn is a three-layer neural network using $tanh$ activation, $\odot$ denotes element-wise product and b_u, b_i are bias terms for user u and item i, respectively.

Graph Convolution Network as Matrix Factorization Model (IDCF-GC). We follow the architecture in GCMC (van den Berg et al., 2017) and adopt graph convolution network for $f_\theta()$. Besides user-specific embedding for user u and item-specific embedding for item i, we consider embeddings for user u's rated items and users who rated on item i, i.e., the one-hop neighbors of u and i in user-item bipartite graph. Denote $\mathcal{N}_{u,m} = \{i | r_{ui} = m\}$ as user u's rated items with rating value m and $\mathcal{N}_{i,m} = \{u | r_{ui} = m\}$ as users who rated on item i with rating value m for $m \geq 1$. Consider graph convolution to aggregate information from neighbors,

$$\begin{aligned} \mathbf{m}_{u,m} =& ReLU(\frac{1}{|\mathcal{N}_{u,m}|} \sum_{i \in \mathcal{N}_{u,m}} \mathbf{W}_{q,m} \mathbf{q}_i), \\ \mathbf{n}_{i,m} =& ReLU(\frac{1}{|\mathcal{N}_{i,m}|} \sum_{u \in \mathcal{N}_{i,m}} \mathbf{W}_{p,m} \mathbf{p}_u), \end{aligned} \tag{12}$$

and combination function: $\mathbf{m}_u = \text{FC}(\{\mathbf{m}_{u,m}\}_m)$, $\mathbf{n}_i = \text{FC'}(\{\mathbf{n}_{i,m}\}_m)$ where FC denotes a fully-connected layer. Then we define the output function

$$\begin{aligned} & f(\mathbf{p}_u, \mathbf{q}_i, \{\mathbf{p}_u\}_{u \in \mathcal{N}_i}, \{\mathbf{q}_i\}_{i \in \mathcal{N}_u}) \\ =& nn'([\mathbf{p}_u \odot \mathbf{q}_i \| \mathbf{p}_u \odot \mathbf{m}_u \| \mathbf{n}_i \odot \mathbf{q}_i \| \mathbf{n}_i \odot \mathbf{m}_u]) + b_u + b_i, \end{aligned} \tag{13}$$

where nn' is a three-layer neural network using $ReLU$ activation function. In Appendix D.1, we provide more details for hyper-parameter setting. Moreover, we specify $l(\hat{r}_{ui}, r_{ui})$ as MSE loss (resp. cross-entropy) for explicit (resp. implicit) feedback data.

5.2. Comparative Results

5.2.1. Interpolation for Few-Shot Users.

Setup. We study the performance on few-shot query users with limited training ratings. We split users in each dataset into two sets: users with more than δ training ratings, denoted as $\mathcal{U}_1$, and those with less than δ training ratings $\mathcal{U}_2$. We basically set $\delta = 30$ for Douban, ML-100K and ML-1M datasets, and $\delta = 20$ for Amazon datasets. For IDCF, we adopt the training algorithm of *inductive learning for interpolation* with $\mathcal{U}_1$ as key users and $\mathcal{U}_2$ as query users.

Table 3. Test AUC and NDCG for few-shot users (FS) and new users (New) in Amazon-Books and Amazon-Beauty.

Method	Amazon-Books				Amazon-Beauty			
	AUC		NDCG		AUC		NDCG	
	Query	New	Query	New	Query	New	Query	New
PMF	0.917	-	0.888	-	0.779	-	0.769	-
NNMF	0.919	-	0.891	-	0.790	-	0.763	-
NGCF	0.916	-	0.896	-	0.793	-	0.775	-
PinSAGE	0.923	-	0.901	-	0.790	-	0.775	-
FISM	-	0.752	-	0.792	-	0.613	-	0.678
MultVAE	-	0.738	-	0.701	-	0.644	-	0.679
IDCF-NN	**0.944**	0.939	**0.928**	0.920	0.792	0.750	**0.783**	0.774
IDCF-GC	0.938	**0.946**	0.921	**0.930**	**0.801**	**0.791**	0.772	**0.791**

We use the training ratings for $\mathcal{U}_1$ and $\mathcal{U}_2$ as input data for our pretraining and adaption, respectively. We compare with several competitors, including 1) powerful transductive methods PMF (Salakhutdinov & Mnih, 2007), NNMF (Dziugaite & Roy, 2015) and GCMC (van den Berg et al., 2017), 2) inductive feature-driven methods NIMC (Zhong et al., 2018), BOMIC (Ledent et al., 2020), and 3) inductive matrix completion model IGMC (Zhang & Chen, 2020). We train each competitor with training ratings of $\mathcal{U}_1$ and $\mathcal{U}_2$.

Table 2 reports RMSE and NDCG for test ratings of all the users and few-shot users in $\mathcal{U}_2$ on Douban, ML-100k and ML-1M. As we can see, IDCF-NN (resp. IDCF-GC) gives very close RMSE and NDCG on all the users and query users to NNMF (resp. GCMC), which suggests that our inductive model can achieve similar reconstruction error and ranking accuracy as corresponding transductive counterpart. The results validate our theoretical analysis in Section 3 that IDCF possesses the same representation capacity as matrix factorization model. Even though IDCF enables inductive representation learning via a parameter-sharing relation model, it does not sacrifice any representation power. Compared with inductive methods, IDCF-GC achieves the best RMSE and NDCG for query users in most cases. The results demonstrate the superiority of IDCF against other feature-driven and inductive matrix completion models.

Table 3 shows AUC and NDCG for test interactions of few-shot users in $\mathcal{U}_2$ on Amazon-Books and Amazon-Beauty. Since Amazon datasets have no user feature, the feature-driven competitors would not work. Also, IGMC would fail to work with implicit feedbacks. We compare with two other graph-based recommendation models, NGCF (Wang et al., 2019) and PinSAGE (Ying et al., 2018). As shown in Table 3, IDCF-NN and IDCF-GC significantly outperform transductive models in implicit feedback setting with $2.3\%/3.0\%$ (resp. $1.0\%/1.0\%$) improvement of AUC/NDCG on Amazon-Books (resps. Amazon-Beauty). The two Amazon datasets are both very sparse with rating density approximately 0.001. One implication here is that our inductive model can provide better performance than transductive models for users with few training ratings.

Table 4. Test RMSE and NDCG for new users on Douban, ML-100K and ML-1M.

Method	Douban		ML-100K		ML-1M	
	RMSE	NDCG	RMSE	NDCG	RMSE	NDCG
NIMC	0.766	0.921	1.089	0.864	1.059	0.883
BOMIC	0.764	0.920	1.088	0.859	1.057	0.879
FISM	1.910	0.824	1.891	0.760	2.283	0.771
MultVAE	2.783	0.823	2.865	0.758	2.981	0.792
IGMC	0.743	-	1.051	-	0.997	-
IDCF-NN	0.749	**0.955**	1.078	0.877	0.994	0.941
IDCF-GC	**0.723**	**0.955**	**1.011**	**0.881**	**0.957**	**0.942**

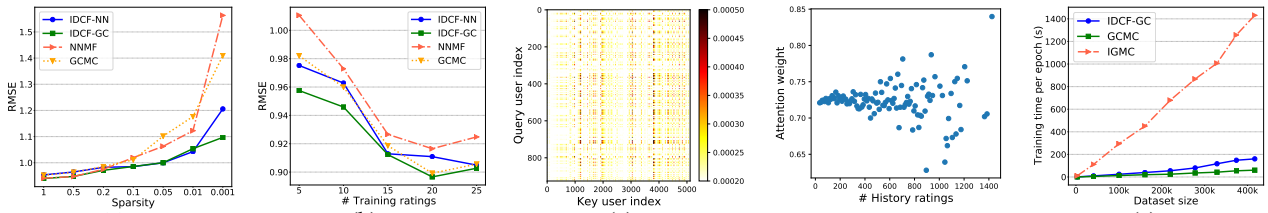
5.2.2. Extrapolation for New Users.

Setup. We then investigate model's generalization performance on new users that are unseen in training. We assume the model is only exposed to the training ratings of $\mathcal{U}_1$ and test its performance on test ratings of $\mathcal{U}_2$. Concretely, for IDCF, we leverage the training algorithm of *inductive learning for extrapolation* with $\mathcal{U}_1$ as both key and query users. The two-stage training uses training ratings of $\mathcal{U}_1$. We compare with inductive models NIMC, BOMIC, IGMC and item-based CF models FISM (Kabbur et al., 2013), MultVAE (Liang et al., 2018).

Table 4 reports results on test ratings for new users in $\mathcal{U}_2$ on Douban, ML-100K and ML-1M. Notably, IDCF-GC outperforms the best competitors by a large margin, with RMSE (resp. NDCG) improvement of 2.6% (resp. 3.6%) on Douban, 3.8% (resp. 2.0%) on ML-100K and 4.0% (resp. 6.7%) on ML-1M. Also, Table 3 reports test AUC and NDCG for new users on two Amazon datasets. The results show that both IDCF-NN and IDCF-GC outperform other competitors with 25.7% (resp. 17.4%) and 22.8% (resp. 16.4%) improvements of AUC (resp. NDCG) on Amazon-Books and Amazon-Beauty, respectively. Such results demonstrate superior power of IDCF for addressing new users in open-world recommendation.

5.3. Further Discussion

Sparse Data and Few-shot Users. A successful recommender system is supposed to handle data sparsity and few-shot users with few historical ratings. Here we construct sparse datasets by using $50\%, 20\%, 10\%, 5\%, 1\%$ and 0.1% training ratings in Movielens-1M, and then compare the test RMSEs of query users in Fig. 3(a). Also, Fig. 3(b) compares the test RMSEs for users with different numbers of historical ratings under 50% sparsity. As shown in Fig. 3(a), as the dataset becomes sparser, the RMSEs of all the models suffer from a drop, but the drop rate of our inductive models IDCF-NN and IDCF-GC is much smaller compared with transductive models NNMF and GCMC. In Fig. 3(b), we find that users with more historical ratings usually have better RMSE scores compared with few-shot users. By contrast, our inductive models IDCF-NN and IDCF-GC exhibit



(a) (b) (c) (d) (e)

Figure 3. Evaluation. (a) Overall RMSE w.r.t # sparsity ratio. (b) User-specific RMSE w.r.t # user's training ratings. (c) Attention weights of query users (y-axis) on key users (x-axis). (d) Key uses' accumulated attention weights w.r.t. # historical ratings. (e) Scalability test.

a more smooth decrease and even outperform other transductive methods NNMF and GCMC for users with very few ratings. In the extreme cases with less than five historical ratings, IDCF-GC achieves 2.5% improvement on RMSE compared with the best transductive method GCMC.

Attention Weight Distribution In Fig. 3(c) we visualize attention weights of IDCF-NN from query users to key users in ML-1M. There is an interesting phenomenon that some of key users appear to be very 'important' and most query users have high attention scores on them. It indicates that the embeddings of these key users are informative and can provide powerful expressiveness for query users' preferences. In Fig. 3(d) we further plots key users' accumulated attention weights (sum of the attention scores over all the query users) w.r.t. # historical ratings. We can see that key users with more historical ratings are more likely to have large attention weights on query users, though they are also more likely to have low attention scores. This observation is consistent with intuition that the users with more historical ratings are easier for the model to identify their interests. Also, the results gives an important hint for selecting useful key users: informative key users are more likely to exsit in users with more historical ratings. In Appendix E, we compare different split ways for key and query users and will provide more discussions on this point.

Scalability Test We further investigate the scalability of IDCF-GC compared with two GNN-based counterparts IGMC and GCMC. We statistic the training time per epoch on ML-1M using a GTX 1080Ti with 11G memory. Here we truncate the dataset and use different numbers of ratings for training. The results are shown in Fig. 3(e). As we can see, when dataset size becomes large, the training times per epoch of three models all exhibit linear increase. IDCF spends approximately one more time than GCMC, while IGMC is approximately ten times slower than IDCF. Nevertheless, while IDCF costs one more training time than GCMC, the latter cannot tackle new unseen users without retraining a model in test stage.

More Insights on IDCF's Effectiveness. The relation model aims at learning graph structures among users that explore more useful proximity information and maximize the benefits of message passing for inductive representation learning. Existing graph-based (recommendation) models rely on a given observed graph, on top of which a GNN model would play as a strong inductive bias. Such inductive bias may be beneficial for some cases (if the graph is fully observed and possesses homophily property) and harmful for other cases (if the graph has noisy/missing links or have heterophily structures). Differently, IDCF enables mutual reinforcement between structure learning and message passing with the guidance of supervised signals from downstream tasks in a fully data-driven manner.

6. Conclusions and Outlook

We have proposed an inductive collaborative filtering framework that learns hidden relational graphs among users to allow effective message passing in the latent space. It accomplishes inductive computation for user-specific representations without compromising on representation capacity and scalablity. Our model achieves state-of-the-art performance on inductive collaborative filtering for recommendation with few-shot and zero-shot users that are commonly encountered in open-world recommendation.

The core idea of IDCF opens a new way for next generation of representation learning, i.e., one can consider a pretrained representation model for one set of existing entities and their representations (through some simple transformations) can be generalized to efficiently compute inductive representations for others, enabling the model to flexibly handle new coming entities in the wild.

Acknowledgement

We would like to thank the anonymous reviewers for their valuable feedbacks and suggestions that help to improve this work. This work was supported by the National Key R&D Program of China [2020YFB1707903], the National Natural Science Foundation of China [61872238,61972250], Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102), the Tencent Marketing Solution Rhino-Bird Focused Research Program [FR202001], and the CCF-Tencent Open Fund [RAGR20200105].

References

Cen, Y., Zhang, J., Zou, X., Zhou, C., Yang, H., and Tang, J. Controllable multi-interest framework for recommendation. In Gupta, R., Liu, Y., Tang, J., and Prakash, B. A. (eds.), *KDD*, pp. 2942–2951, 2020.

Chen, Y., Wu, L., and Zaki, M. J. Iterative deep graph learning for graph neural networks: Better and robust node embeddings. In *NeurIPS*, 2020.

Cheng, H., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. Wide & deep learning for recommender systems. In *DLRS*, pp. 7–10, 2016.

Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In *RecSys*, pp. 191–198, 2016.

Cremonesi, P., Koren, Y., and Turrin, R. Performance of recommender algorithms on top-n recommendation tasks. In *RecSys*, pp. 39–46. ACM, 2010.

Dziugaite, G. K. and Roy, D. M. Neural network matrix factorization. *CoRR*, abs/1511.06443, 2015.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *ICML*, pp. 1263–1272, 2017.

Hamilton, W. L., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. In *NeurIPS*, pp. 1024–1034, 2017.

Hartford, J. S., Graham, D. R., Leyton-Brown, K., and Ravanbakhsh, S. Deep models of interactions across sets. In *ICML*, pp. 1914–1923, 2018.

Hu, Y., Koren, Y., and Volinsky, C. Collaborative filtering for implicit feedback datasets. In *ICDM*, pp. 263–272, 2008.

Jain, P. and Dhillon, I. S. Provable inductive matrix completion. *CoRR*, abs/1306.0626, 2013.

Kabbur, S., Ning, X., and Karypis, G. FISM: factored item similarity models for top-n recommender systems. In *KDD*, pp. 659–667. ACM, 2013.

Kang, W., Cheng, D. Z., Chen, T., Yi, X., Lin, D., Hong, L., and Chi, E. H. Learning multi-granular quantized embeddings for large-vocab categorical features in recommender systems. In *WWW*, pp. 562–566, 2020.

Khalil, E. B., Dai, H., Zhang, Y., Dilkina, B., and Song, L. Learning combinatorial optimization algorithms over graphs. In *NeurIPS*, pp. 6348–6358, 2017.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009.

Ledent, A., Alves, R., and Kloft, M. Orthogonal inductive matrix completion. *CoRR*, abs/2004.01653, 2020.

Lee, H., Im, J., Jang, S., Cho, H., and Chung, S. Melu: Meta-learned user preference estimator for cold-start recommendation. In *KDD*, pp. 1073–1082, 2019.

Liang, D., Krishnan, R. G., Hoffman, M. D., and Jebara, T. Variational autoencoders for collaborative filtering. In Champin, P., Gandon, F. L., Lalmas, M., and Ipeirotis, P. G. (eds.), *WWW*, pp. 689–698. ACM, 2018.

Liu, H., Lu, J., Zhao, X., Xu, S., Peng, H., Liu, Y., Zhang, Z., Li, J., Jin, J., Bao, Y., and Yan, W. Kalman filtering attention for user behavior modeling in CTR prediction. In *NeurIPS*, 2020.

Ma, J., Zhou, C., Cui, P., Yang, H., and Zhu, W. Learning disentangled representations for recommendation. In *NeurIPS*, pp. 5712–5723, 2019.

Manchanda, S., Mittal, A., Dhawan, A., Medya, S., Ranu, S., and Singh, A. K. Learning heuristics over large graphs via deep reinforcement learning. *CoRR*, abs/1903.03332, 2019.

Mikolov, T., Chen, K., Corrado, G., and Dean, J. Efficient estimation of word representations in vector space. In *ICLR, Workshop*, 2013.

Mohri, M., Rostamizadeh, A., and Talwalkar, A. *Foundations of Machine Learning*. Adaptive computation and machine learning. MIT Press, 2012.

Monti, F., Bronstein, M. M., and Bresson, X. Geometric matrix completion with recurrent multi-graph neural networks. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *NeurIPS*, pp. 3697–3707, 2017.

Qian, T., Liang, Y., and Li, Q. Solving cold start problem in recommendation with attribute graph neural networks. *CoRR*, abs/1912.12398, 2019.

Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. BPR: bayesian personalized ranking from implicit feedback. In *UAI*, pp. 452–461, 2009.

Salakhutdinov, R. and Mnih, A. Probabilistic matrix factorization. In Platt, J. C., Koller, D., Singer, Y., and Roweis, S. T. (eds.), *NeurIPS*, pp. 1257–1264, 2007.

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The graph neural network model. *IEEE Trans. Neural Networks*, 20(1):61–80, 2009.

Sedhain, S., Menon, A. K., Sanner, S., and Xie, L. Autorec: Autoencoders meet collaborative filtering. In *WWW (Companion Volume)*, pp. 111–112. ACM, 2015.

Srebro, N., Alon, N., and Jaakkola, T. S. Generalization error bounds for collaborative prediction with low-rank matrices. In *NeurIPS*, pp. 1321–1328, 2004.

van den Berg, R., Kipf, T. N., and Welling, M. Graph convolutional matrix completion. *CoRR*, abs/1706.02263, 2017.

Wang, X., He, X., Wang, M., Feng, F., and Chua, T. Neural graph collaborative filtering. In *SIGIR*, pp. 165–174, 2019.

Xu, M., Jin, R., and Zhou, Z. Speedup matrix completion with side information: Application to multi-label learning. In *NeurIPS*, pp. 2301–2309, 2013.

Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., and Leskovec, J. Graph convolutional neural networks for web-scale recommender systems. In Guo, Y. and Farooq, F. (eds.), *SIGKDD*, pp. 974–983. ACM, 2018.

Zhang, M. and Chen, Y. Inductive matrix completion based on graph neural networks. In *ICLR*, 2020.

Zheng, Y., Tang, B., Ding, W., and Zhou, H. A neural autoregressive approach to collaborative filtering. In *ICML*, pp. 764–773, 2016.

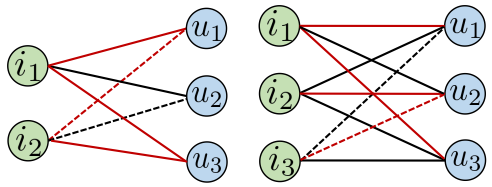
Zhong, K., Song, Z., Jain, P., and Dhillon, I. S. Nonlinear inductive matrix completion based on one-layer neural networks. *CoRR*, abs/1805.10477, 2018.

Appendix

A. Discussion on Representation Power and Expressiveness

We compare IDCF with other related models from two perspectives in order to shed more lights on the advantages and differences of our model.

Comparison of Representation Power. We first provide a comparison with related works on methodological level as a clear elaboration for essential differences of our work to others. In Fig. 5, we present an intuitive comparison with general CF model, local-graph-based inductive CF and item-based CF model. As shown in Fig. 5(a), general collaborative filtering assumes user-specific embeddings for users and learn them collaboratively among all the users in one dataset. It disables inductive learning due to such learnable embeddings. Item-based model, as shown in Fig. 5(b), leverages embeddings of user's historically rated items to compute user's embeddings via some pooling methods. The learnable parameters only lie in the item space. It suffers from limited representation capacity compared to general CF model that assumes both user and item embeddings. Besides, local-graph-based inductive model (e.g. (Zhang & Chen, 2020)), shown in Fig. 5(b), extracts local subgraph structures within 1-hop neighbors of each user-item pair (i.e., rated items of the user and users who rated the item) from a bipartite graph of all the observed user-item ratings and use GNNs to encode such graph structures for rating prediction. Note that the model requires that the local subgraphs do not contain user and item indices, so it cannot output user-specific embeddings. Its representation power is limited since it cannot represent diverse user preferences with arbitrary rating history on items. Also it cannot output user representations. Differently, our model IDCF, as shown in Fig. 5(d) adopts item-based embedding as initial states for query users to compute attention scores on key users and aggregate the embeddings (i.e., meta latents) of key users to estimate user-specific embeddings for query users, which maintains ability to produce user representations with enough representation power and meanwhile achieves inductive learning.



(a) (b)

Figure 4. Comparison of expressiveness. Feature-driven and local-graph-based models fail in (a) and (b), respectively. IDCF works effectively in both cases with superior expressiveness.

Comparison of Expressiveness. We provide a comparison with feature-driven and local-graph-based inductive (matrix completion) models through two cases in Fig. 4 so as to highlight the superior expressiveness of IDCF. Here we assume ratings are within $\{-1, 1\}$ (positive, denoted by red line, and negative, denoted by black line). The solid lines are observed ratings for training and dash lines are test ratings. In Fig. 4(a), we consider test ratings (u_1, i_2) and (u_2, i_2). For local-graph-based models, the 1-hop local subgraphs of (u_1, i_2) (resp. (u_2, i_2)) consists of $\{i_1, i_2, u_1, u_3\}$ (resp. $\{i_1, i_2, u_2, u_3\}$) and the subgraph structures are different for two cases due to the positive (resp. negative) edge for (u_1, i_1) (resp. (u_2, i_1)). The local-graph-based models can give right prediction for two ratings relying on different structures. Also, CF models and IDCF can work smoothly in this case, relying on different rating history of u_1 and u_2. However, feature-driven models will fail once u_1 and u_2 have the same features though two users have different rating history. In Fig. 4(b), we consider test ratings (u_1, i_3) and (u_2, i_3). The 1-hop local subgraphs of (u_1, i_3) (resp. (u_2, i_3)) consists of $(i_1, i_2, i_3, u_1, u_3)$ (resp. $(i_1, i_2, i_3, u_2, u_3)$) and the subgraph structures are the same. Thus, the local-graph-based models will fail to distinguish two inputs and give the same prediction for (u_1, i_3) and (u_2, i_3). Differently, CF models and IDCF can recognize that u_3 has similar rating patterns with u_1 and different from u_2, thus pushing the embedding of u_1 (resp. u_2) close to (resp. distant from) u_3, which guides the model to right prediction. Note that the first case becomes a common issue when the feature space is small while the second case becomes general when the rating patterns of users are not distinct enough throughout a dataset, which induces similar local subgraph structures. Therefore, IDCF enjoys superior expressiveness for input data than feature-driven and local-graph-based inductive (matrix completion) models. Also, it maintains as good expressiveness as (transductive) CF models.

B. Proofs in Section 3

B.1. Proof of Theorem 1

Proof. The proof is trivial by construction. Assume the optimal $\mathbf{P}_2$ for Eq. (3) as $\mathbf{P}_2^*$. Since $\mathbf{P}_1$ given by Eq. (1) is column-full-rank, for any column vector $\mathbf{p}_{u'}^*$ in $\mathbf{P}_2^*$ ($u' \in \mathcal{U}_2$), there exists $\mathbf{c}_{u'}^*$ such that ${\mathbf{c}_{u'}^*}^\top \mathbf{P}_1 = \mathbf{p}_{u'}^*$. Hence, $\mathbf{C}^* = [\mathbf{c}_{u'}^*]_{u' \in \mathcal{U}_2}$ is a solution for Eq. (2) and gives $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_2, R_2) < \epsilon$. □

B.2. Proof of Theorem 2

Proof. With fixed a true rating matrix R_2 to be learned and a probability distribution $\mathcal{P}$ over $[M_q] \times [N]$, which is unknown to the learner, we consider the problem under the framework of standard PAC learning. We can treat the

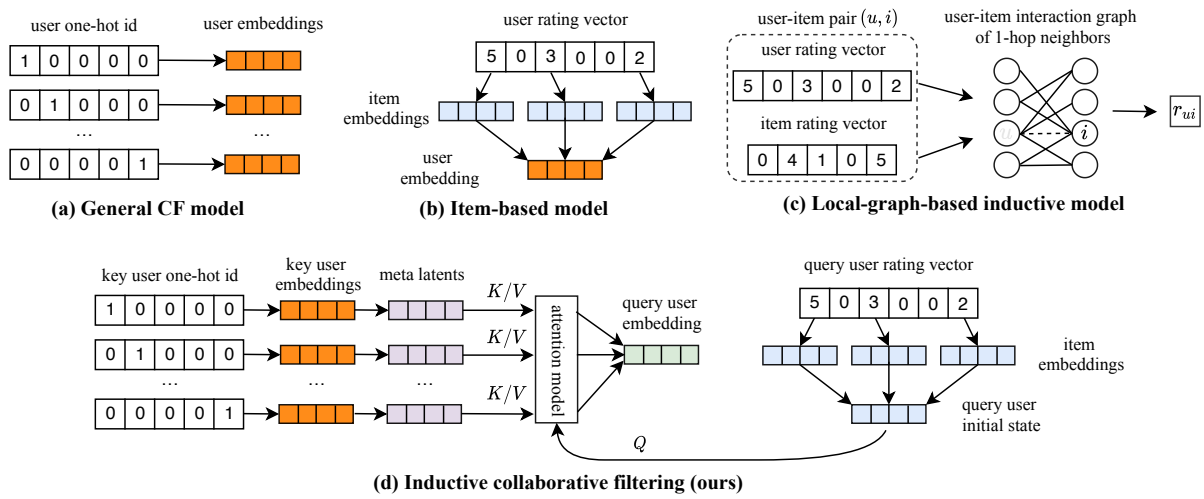


Figure 5. Comparison with related works on methodological level. We highlight the advantage of proposed model IDCF: it can 1) inductively compute user representations (or embeddings) and meanwhile 2) maintain enough capacity as general CF models.

matrix R_2 as a function $(u', i) \to r_{u'i}$. Let $\mathcal{R}$, a set of matrices in $\mathbb{R}^{M_q \times N}$, denotes the hypothesis class of this problem. Then the input to the learner is a sample of R_2 denoted as

$$\mathcal{T} = \left((u'_t, i_t, r_{u'_t i_t}) | (u'_t, i_t) \in \mathcal{S}_q\right),$$

where $\mathcal{S}_q = \{(u'_t, i_t)\} \in ([M_q] \times [N])^{T_2}$ is a set with size T_2 containing indices of the observed entries in R_2 and each (u', i) in $\mathcal{S}_q$ is independently chosen according to the distribution $\mathcal{P}$. When using $\mathcal{T}$ as training examples for the learner, it minimizes the error $\mathcal{D}_{\mathcal{S}_q}(\hat{R}_2, R_2) = \frac{1}{T_2}\sum_{(u',i)\in\mathcal{S}_q} l(r_{u'i}, \hat{r}_{u'i})$. We are interested in the generalization error of the learner, which is defined as

$$\mathcal{D}(\hat{R}_2, R_2) = \mathbb{E}_{(u',i)\in\mathcal{P}}[l(r_{u'i}, \hat{r}_{u'i})].$$

The (empirical) Rademacher complexity of $\mathcal{R}$ w.r.t. the sample $\mathcal{T}$ is defined as

$$Rad_{\mathcal{T}}(\mathcal{R}) = \frac{1}{T_2}\mathbb{E}_\sigma\left[\sup_{\hat{R}_2\in\mathcal{R}} \sum_{t=1}^{T_2} \sigma_t \hat{r}_{u'_t i_t}\right],$$

where $\sigma_t \in \{-1, 1\}$ is a random variable with probability $Pr(\sigma_t = 1) = Pr(\sigma_t = -1) = \frac{1}{2}$. Assume $l(\cdot,\cdot)$ is L-Lipschitz w.r.t. the first argument and $|l(\cdot,\cdot)|$ is bounded by a constant. Then a general result for generalization bound of $\mathcal{R}$ is

Lemma 1. *(Generalization bound (Mohri et al., 2012)): For a sample $\mathcal{T}$ with random choice of $\mathcal{S}_q = ([M_q]\times[N])^{T_2}$, it holds that for any $\hat{R}_2 \in \mathcal{R}$ and confidence parameter $0 < \delta < 1$,*

$$Pr(\mathcal{D}(\hat{R}_2, R_2) \le \mathcal{D}_{\mathcal{S}_q}(\hat{R}_2, R_2) + G) \ge 1 - \delta, \quad (14)$$

where,

$$G = 2L \cdot Rad(\mathcal{R}) + O\left(\sqrt{\frac{\ln(1/\delta)}{T_2}}\right).$$

Based on the lemma, we need to further estimate the Rademacher complexity in our model to complete the proof. In our model, $\hat{R}_2 = \mathbf{C}^\top \mathbf{P}_k \mathbf{Q}$ and the entry $\hat{r}_{u'i}$ is given by $\hat{r}_{u'i} = \mathbf{p}_{u'}^\top \mathbf{q}_i = \mathbf{c}_{u'}^\top \mathbf{P}_k \mathbf{q}_i$ (where $\mathbf{c}_{u'}$ is the u'-th colunm vector of $\mathbf{C}$). Define $\mathcal{C}$ as a set of matrices,

$$\mathcal{C} = \{\mathbf{A} \in [0,1]^{M_q \times M_k} : \|\mathbf{a}_{u'}\|_1 = \sum_{u=1}^{M_k} |a_{u'u}| = 1\}.$$

Then we have

$$\begin{aligned}
T_2 \cdot Rad_{\mathcal{T}}(\mathcal{R}) &= \mathbb{E}_\sigma\left[\sup_{\mathbf{C}\in\mathcal{C}} \sum_{t=1}^{T_2} \sigma_t \mathbf{c}_{u'_t}^\top \mathbf{P}_k \mathbf{q}_{i_t}\right] \\
&= \mathbb{E}_\sigma\left[\sup_{\mathbf{C}\in\mathcal{C}} \sum_{u'=1}^{M_q} \mathbf{c}_{u'}^\top \cdot \left(\sum_{t:u_t=u'} \sigma_t \hat{R}_{k,*i_t}\right)\right] \\
&\quad (\text{since } \hat{R}_{k,*i_t} = \mathbf{P}_k \mathbf{q}_{i_t}) \\
&\le H \cdot \mathbb{E}_\sigma\left[\sup_{\mathbf{A}\in\mathcal{A}} \sum_{u'=1}^{M_q} \mathbf{a}_{u'}^\top \cdot \left(\sum_{t:u_t=u'} \sigma_t \hat{R}_{k,*i_t}\right)\right] \\
&= H \cdot \mathbb{E}_\sigma\left[\sum_{u'=1}^{M_q} \max_{u\in[M_k]} \left(\sum_{t:u_t=u'} \sigma_t \hat{r}_{ui_t}\right)\right].
\end{aligned} \quad (15)$$

The last equation is due to the fact that $\mathbf{a}_{u'}$ is a probability distribution for choosing entries in $R_{k,*i_t}$, the i_t-th column of matrix $\hat{R}_k$. In fact, we can treat the $\max_{u\in[M_k]}$ inside

the sum over all $u' \in \mathcal{U}_2$ as a mapping κ from $u' \in [M_q]$ to $u \in [M_k]$. Let $\mathcal{K} = \{\kappa : [M_q] \to [M_k]\}$ be the set of all mappings from $[M_q]$ to $[M_k]$, and then the above formula can be written as

$$\mathbb{E}_\sigma \left[\sum_{u'=1}^{M_q} \max_{u \in [M_k]} \left(\sum_{t:u_t=u'} \sigma_t \hat{r}_{ui_t} \right) \right] \tag{16}$$

$$= \mathbb{E}_\sigma \left[\sup_{\kappa \in \mathcal{K}} \sum_{u'=1}^{M_q} \sum_{t:u_t=u'} \sigma_t \hat{r}_{\kappa(u'),i_t} \right] \tag{17}$$

$$= \mathbb{E}_\sigma \left[\sup_{\kappa \in \mathcal{K}} \sum_{t=1}^{T_2} \sigma_t \hat{r}_{\kappa(u_t),i_t} \right] \tag{18}$$

$$\leq B\sqrt{T_2} \cdot \sqrt{2M_q \log M_k}. \tag{19}$$

The last inequality is according to the Massart Lemma. Hence, we have

$$Rad_{\mathcal{T}}(\mathcal{R}) \leq HB\sqrt{\frac{2M_q \log M_k}{T_2}}. \tag{20}$$

Incorporating Eq. (20) into Eq. (14), we will arrive at the result in this theorem. □

C. Extensions of IDCF

IDCF can be extended to feature-based setting and deal with extreme cold-start recommendation where test users have no historical ratings. Here, we provide details of feature-based IDCF (IDCF-HY) which indeed is a hybrid model that considers both user features and one-hot user indices. Furthermore, we discuss in the views of transfer-learning and meta-learning that can be leveraged to enhance our framework as future study.

C.1. Hybrid Model with Features (IDCF-HY)

Assume $\mathbf{a}_u$ denotes user u's raw feature vector, i.e., a concatenation of all the features (often including binary, categorical and continuous variables) where categorical features can be denoted by one-hot or multi-hot vectors. If one has m user features in total, then $\mathbf{a}_u$ can be

$$\mathbf{a}_u = [\mathbf{a}_{u1}||\mathbf{a}_{u2}||\mathbf{a}_{u3}||\cdots||\mathbf{a}_{um}].$$

Then we consider user-sharing embedding function $\mathbf{y}_i()$ which can embed each feature vector into a d-dimensional embedding vector:

$$\mathbf{y}_u = [\mathbf{y}_1(\mathbf{a}_{u1})||\mathbf{y}_2(\mathbf{a}_{u2})||\mathbf{y}_3(\mathbf{a}_{u3})||\cdots||\mathbf{y}_m(\mathbf{a}_{um})].$$

Similarly, for item feature $\mathbf{b}_i = [\mathbf{b}_{i1}||\mathbf{b}_{i2}||\mathbf{b}_{i3}||\cdots||\mathbf{b}_{in}]$, we have its embedding representation:

$$\mathbf{z}_i = [\mathbf{z}_1(\mathbf{b}_{i1})||\mathbf{z}_2(\mathbf{b}_{i2})||\mathbf{z}_3(\mathbf{b}_{i3})||\cdots||\mathbf{z}_n(\mathbf{b}_{in})].$$

Also, we assume user-specific index embedding $\mathbf{p}_u$ and item-specific index embedding $\mathbf{q}_i$ for user u and item i, respectively, as is in Section 3. The prediction for user u's rating on item i can be

$$\hat{r}_{ui} = g_\theta(\mathbf{p}_u, \mathbf{y}_u, \mathbf{q}_i, \mathbf{z}_i), \tag{21}$$

where g_θ can be a shallow neural network with parameters denoted by θ. To keep notation clean, we denote $\mathbf{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \cdots, \mathbf{y}_m\}$ and $\mathbf{Z} = \{\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_n\}$. Then for key users in $\mathcal{U}_k$ with rating matrix R_k, we consider the optimization problem,

$$\min_{\mathbf{P}_k, \mathbf{Q}, \mathbf{Y}, \mathbf{Z}, \theta} \mathcal{D}_{\mathcal{S}_k}(\hat{R}_k, R_k), \tag{22}$$

based on which we get learned feature embedding functions $\mathbf{Y}$, $\mathbf{Z}$ as well as transductive embedding matrices $\mathbf{P}_k$, $\mathbf{Q}$ which we further use to compute inductive embeddings for query users.

For query users, feature embeddings can be obtained by the learned $\mathbf{Y}$ and $\mathbf{Z}$ in Eq. (22), i.e., $\mathbf{y}_{u'} = [\mathbf{y}_{u'1}(\mathbf{a}_{u'1})||\cdots||\mathbf{y}_{u'm}(\mathbf{a}_{u'm})]$ where $\mathbf{a}_{u'}$ is raw feature vector of user u'. Then we have a relation learning model h_w that consists of a multi-head attention function and use user feature as input $\mathbf{d}_{u'} = \mathbf{y}_{u'}$. The inductive user-specific representation can be given by $\mathbf{p}_{u'} = h_w(\mathbf{d}_{u'})$ (i.e., Eq. (5) and Eq. (6)), similar as the CF setting in Section 3. The rating of user u' on item i can be predicted by $\hat{r}_{u'i} = g_\theta(\mathbf{p}_{u'}, \mathbf{y}_{u'}, \mathbf{q}_i, \mathbf{z}_i)$. Also, the optimization for inductive relation model is

$$\min_{w,\theta} \mathcal{D}_{\mathcal{S}_q}(\hat{R}_q, R_q). \tag{23}$$

C.2. Extreme Cold-Start Recommendation

For cold-start recommendation where test users have no historical rating, we have no information about users if without any side information. In such case, most CF models would fail for personalized recommendation and degrade to a trivial one which outputs the same result (or the same distribution) to all the users using the popularity of items. For IDCF, the set $\mathcal{I}_{u'}$ in Eq. (4) would be empty for users with no historical rating, in which situation we can randomly select a group of key users to construct $\mathcal{I}_{u'}$ used for computing attentive scores with key users. Another method is to directly use average embeddings of all the key users as estimated embeddings for query users. In such case, the model degrades to ItemPop (using the numbers of users who rated the item for prediction).

On the other hand, if side information (such as user profile features) is available, our hybrid model IDCF-HY can leverage user features for computing inductive embeddings, which enables extreme cold-start recommendation. We apply this method to cold-start recommendation on Movielens-1M using features and the results are given in Appendix E.

C.3. Transfer Learning & Meta-Learning

Another extension of IDCF is to consider transfer learning on cross-domain recommendation tasks or when treating recommendation for different users as different tasks like (Lee et al., 2019). Transfer learning and meta learning have shown power in learning generalizable models that can adapt to new tasks. In our framework, we can also take advantage of transfer learning (few-shot learning or zero-shot learning) or mete-learning algorithms to train our relation learning model h_w. For example, if using model-agnostic meta-learning algorithm for the second-stage optimization, we can first compute one-step (or multi-step) gradient update independently for each user (or a group of clustering users) in a batch and then average them as one global update for the model. The meta-learning can be applied over different groups of users or cross-domain datasets.

D. Details in Implementations

We provide implementation details that are not presented in Section 5 in order for reproducibility.

D.1. Hyper-parameter Settings

We present details for hyper-parameter settings in different datasets. We use $L = 4$ attention heads for our inductive relation learning model among all the datasets. For Douban and ML-100K, each attention head randomly samples 200 key users for computing attention weights. For ML-1M, we set sample size as 500; for Amazon-Books and Amazon-Beauty, we set it as 2000. We use Adam optimizer and learning rates are searched within [0.1, 0.01, 0.001, 0.0001]. For pretraining, we consdier L2 regularization for user and item embeddings. The regularization weights are searched within [0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2]. The mini-batch sizes are searched within [64, 256, 512, 1024, 2048] to keep a proper balance between training efficiency and performance. For adaption stage, regularization weight λ is set as 10 for five datasets. Besides, different hyper-parameters for architectures are used in three implementations.

IDCF-NN. For Douban and ML-100K, we use embedding dimension $d = 16$ and neural size $48 - 32 - 32 - 1$ for f_θ. For ML-1M, ML-10M and Amazon-Books, we use $d = 32$ and neural size $96 - 64 - 64 - 1$ for f_θ.

IDCF-GC. For Douban and ML-100K, we use embedding dimension $d = 32$ and neural size $128 - 32 - 32 - 1$ for f_θ. For ML-1M, ML-10M and Amazon-Books, we use $d = 64$ and neural size $256 - 64 - 64 - 1$ for f_θ.

IDCF-HY. We use embedding size $d = 32$ for each feature in ML-1M as well as user-specific and item-specific index embeddings. The neural size of g_θ is set as $320-64-64-1$.

D.2. Evaluation Metrics

We provide details for our adopted evaluation metrics. In our experiments, we follow evaluation protocols commonly used in previous works in different settings. Three metrics used in our paper are as follows.

- **RMSE:** Root Mean Square Error is a commonly used metric for explicit feedback data and measures the averaged L2 distance between predicted ratings and ground-truth ratings:

$$RMSE = \sqrt{\frac{\sum_{(u,i)\in\mathcal{I}^+} (\hat{r}_{ui} - r_{ui})^2}{|\mathcal{I}^+|}}. \tag{24}$$

- **AUC:** Area Under the ROC Curve is a metric for implicit feedback data. It measures general consistency between a ranking list of predicted scores and ground-truth ranking with 1's before 0's. More specifically, AUC counts the average area under the curve of true-positive v.s. false-positive curve:

$$AUC = \frac{\sum_{(u,i)\in\mathcal{I}^+} \sum_{(u',i')\in\mathcal{I}^-} \delta(\hat{r}_{u,i} > \hat{r}_{u',i'})}{|\mathcal{I}^+||\mathcal{I}^-|}, \tag{25}$$

  where $\mathcal{I}^+ = \{(u,i)|r_{ui} > 0\}$ and $\mathcal{I}^- = \{(u',i')|r_{u'j'} = 0\}$ denote the sets of observed user-item interaction pairs and unobserved user-item pairs respectively. The indicator $\delta(\hat{r}_{ui} > \hat{r}_{uj})$ returns 1 when $\hat{r}_{ui} > \hat{r}_{uj}$ and 0 otherwise. Since we only have ground-truth positive examples (clicked items) for users, we negatively sample five items as negative examples (non-clicked items) for each user-item rating in dataset, which composes the set $\mathcal{I}^-$.

- **NDCG:** Normalized discounted cumulative gain (Normalized DCG) measures the usefulness, or gain, of a recommended item based on its position in the result list. NDCG can be used to evaluate the model on both explicit and implicit feedback data. The gain is accumulated from the top of the recommendation list to the bottom, with the gain of each result discounted at lower ranks. The NDCG metric is computed for each user and can measures the averaged performance of personalized recommendation. Given the ranking list of recommended items to a user u, denoted as $\hat{\mathcal{K}}_u$, its DCG is defined as:

$$DCG = \sum_{i\in\hat{\mathcal{K}}_u} \frac{rel_i}{\log_2(i+1)}. \tag{26}$$

  where rel_i is the graded relevance of item i with user u. For explicit feedbacks, rel_i is the ground-truth rating of user u on item i. For implicit feedbacks, $rel_i = 1$ for

observed user-item interaction and $rel_i = 0$ otherwise. The normalized discounted cumulative gain, or NDCG, is computed as:

$$NDCG@K = \frac{DCG@K}{IDCG@K} \tag{27}$$

where $IDCG@K$ is ideal discounted cumulative gain: $IDCG@K = \sum_{i \in \mathcal{K}_u} \frac{rel_i}{\log_2(i+1)}$, and $\mathcal{K}_u$ represents the ground-truth ranking list of relevant items (ordered by their ground-truth ratings/interactions by user u).

E. More Experiment Results

E.1. Impacts of different splits for key and query users

In our experiments in Section 5, we basically consider users with more than δ training ratings as $\mathcal{U}_1$ and the remaining as $\mathcal{U}_2$, based on which we construct key users and query users to study model's performance on few-shot query users (for inductive interpolation) and new unseen users (for inductive extrapolation). Here we provide a further discussions on two spliting ways and study the impact on model performance.

- **Threshold**: we select users with more than δ training ratings as $\mathcal{U}_1$ and users with less than δ training ratings as $\mathcal{U}_2$.
- **Random**: we set a ratio $\gamma \in (0, 1)$ and randomly sample $\gamma \times 100\%$ of users in the dataset as $\mathcal{U}_1$. The remaining users are grouped as $\mathcal{U}_2$.

We consider $\delta = [20, 30, 40, 50, 60, 70]$ and $\gamma = [0.97, 0.85, 0.75, 0.68, 0.62, 0.57]$ (which exactly gives the same ratio of $|\mathcal{U}_1|$ and $|\mathcal{U}_2|$ as corresponding δ in threshold split[5]) in Movielens-1M dataset. For each spliting, we also consider two situations for key users and query users: 1) inductive learning for interpolation on few-shot query users, i.e., the first-starge training is on the training ratings of key users $\mathcal{U}_k = \mathcal{U}_1$ and the second-stage training is on the training ratings of query users $\mathcal{U}_k = \mathcal{U}_2$; inductive learning for extrapolation on zero-shot new users, i.e., the two-stage trainings are both on the training ratings of a same group of users $\mathcal{U}_k = \mathcal{U}_q = \mathcal{U}_1$. We test the model on the testing ratings of users in $\mathcal{U}_2$. The results of IDCF-NN are presented in Table 5 where we report test RMSEs on all the users, few-shot query users and zero-shot new users.

As we can see from Table 5, with *threshold split*, as δ increases (we have fewer key users and more query users and they both have more training ratings on average), test RMSEs for query users exhibit a decrease. The reason is two-folds: 1) since key usres have more training ratings, the transductive model can learn better representations; 2) since query users have more training ratings, the inductive model would have better generalization ability. On the other hand, with different spliting thresholds, test RMSEs for new users remain in a fixed level. The results demonstrate that the performance of IDCF on new unseen users is not sensitive to different splitting thresholds. However, with *random split*, when γ decreases (also we have fewer key users and more query users but their average training ratings stay unchanged), RMSEs for new users suffer from an obvious decrease. One possible reason is that when we use smaller ratio of key users with random split, the 'informative' key users in the dataset are more likely to be ignored. (Recall that, as is shown in Fig. 3(c), there exist some important key users that give high attention weights on query users.) If such key users are missing, the performance would be affected due to insufficient expressive power of the inductive representation model.

Comparing threshold split with random split, we can find that when using the same ratio of key users and query users (i.e., the same column in Table 5), RMSEs on new users with threshold split are always better than those with random split. Such observation again shows that key users with more historical ratings would be more informative for providing useful information to inductive representation learning on query users, and again echo the results in Fig. 3(d) which demonstrates that important key users who give large attention weights on query users tend to exist in users with sufficient historical ratings.

E.2. Ablation Studies

In Table 6 we present the results of ablation study on ML-1M and Amazon-Books datasets. We compare IDCF-GC with 1) RD-Item (using randomized item embeddings), 2) Trans-User (directly optimizing $\mathbf{P}_q$ in Eqn. (2)) and 3) Meta-Path (using meta-path *user-item-user* in the observed user-item bipartite graph to determine users' neighbors for message passing). The results show that RD-Item performs much worse than IDCF-GC since the randomized item embeddings may provide wrong signals for both graph learning and final prediction. Compared with Trans-User, IDCF-GC significantly outperforms it over a large margin. The reason is that directly optimizing $\mathbf{P}_q$ would lead to serious overfitting since query users have few training data. Furthermore, we can see that Meta-Path provides inferior performance than IDCF-GC in Table 1. The reason is that meta-path can only identify limited relations from observed bipartite graph that often has missing/noisy links, while IDCF learns and explores useful semantic relations for sufficient message passing.

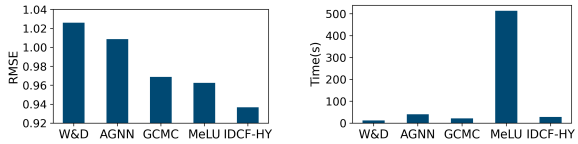
[5] For example, using $\gamma = 0.97$ in random split will result in the same sizes of $\mathcal{U}_1$ and $\mathcal{U}_2$ as using $\delta = 20$ in threshold split.

Table 5. Test RMSEs on all the users (All), few-shot query users (FS) and new users (New) of IDCF-NN in Movielens-1M using different splits for key and query users. (Lower RMSE is better)

	δ	20	30	40	50	60	70
Threshold	All (RMSE)	0.8440	0.8437	0.8439	0.8440	0.8444	0.8451
	FS (RMSE)	0.9785	0.9525	0.9213	0.9166	0.9202	0.9160
	New (RMSE)	0.9945	0.9942	0.9902	0.9883	0.9911	0.9929
Random	γ	0.97	0.85	0.75	0.68	0.62	0.57
	All (RMSE)	0.8446	0.8536	0.8587	0.8637	0.8669	0.8689
	FS (RMSE)	0.8863	0.8848	0.8760	0.8805	0.8824	0.8855
	New (RMSE)	0.9901	0.9923	0.9956	0.1001	1.0198	1.0262

Table 6. Ablation studies on ML-1M and Amazon-Books datasets. (Lower RMSE and Higher AUC/NDCG are better)

Method	**ML-1M**				**Amazon-Books**			
	Query		**New**		**Query**		**New**	
	RMSE	**NDCG**	**RMSE**	**NDCG**	**AUC**	**NDCG**	**AUC**	**NDCG**
IDCF-GC	**0.944**	**0.940**	**0.957**	**0.942**	**0.938**	**0.946**	**0.921**	**0.930**
RD-Item	1.014	0.843	1.023	0.835	0.665	0.701	0.832	0.821
Trans-User	0.992	0.876	-	-	0.845	0.821	-	-
Meta-Path	0.959	0.912	0.981	0.892	0.910	0.916	0.882	0.901



(a) Test RMSEs (b) Training time per epoch

Figure 6. Performance comparison for extreme cold-start recommendation on ML-1M with user profile features.

E.3. Cold-Start with User Features

We also wonder if our inductive model can handle extreme cold-start users with no historical rating [6]. Note that cold-start users are different and more challenging compared to new (unseen) users. For new users, the model can still use historical ratings as input features during inference, though it cannot be trained on these ratings. To enable cold-start recommendation, we leverage user attribute features in Movielens-1M. We use the dataset provided by (Lee et al., 2019), which contains attribute features and split warm-start and cold-start users. For IDCF, we also adopt the training algorithm of inductive learning for extrapolation and treat the warm-start users as key and query users. We user the *warm-start* users' training ratings for model training and the *cold-start* users' test ratings for test. We compare with Wide&Deep network (Cheng et al., 2016), GCMC (using feature vectors) and two recently proposed methods for cold-start recommendation: graph-based model AGNN (Qian et al., 2019) and meta-learning model MeLU (Lee et al., 2019).

[6]In some literature, cold-start users also mean users with few historical ratings (for training or/and inference). Here we consider extreme cold-start recommendation for users with no historical rating for both training and inference.

Fig. 6(a) gives the test RMSEs for all the models. It shows that our IDCF-HY outperforms the competitors, achieving 2.6% improvement of RMSE over the best one MeLU even on the difficult zero-shot recommendation task. The result indicates that IDCF is a promising approach to handle new users with no historical behavior in real-world dynamic systems. We also compare the training time per epoch of each method in Fig. 6(b). By contrast, IDCF-HY is much faster than MeLU and as efficient as AGNN and GCMC.